

PROJECT DOCUMENTATION

CloudNative-App-Deployment

End-to-End Cloud-Native CI/CD Pipeline on AWS EKS with Docker, Jenkins, Terraform, Kubernetes, Prometheus & Grafana

Field	Value
Application	Trendify
Repository	github.com/Akshaya33-Devops/CloudNative-App-Deployment
Author	Akshaya M
Cloud Region	AWS ap-south-1 (Mumbai)
Document Date	20-09-2026

1. Project Overview

1.1 What the project does

CloudNative-App-Deployment takes a static front-end application (Trendify, a Vite-built e-commerce UI) and turns it into a fully automated, production-style cloud deployment. The goal was not just to “get the app online,” but to reproduce, at small scale, the same pipeline a real DevOps team would run: code goes into version control, a CI/CD server builds and ships it automatically, infrastructure is defined as code rather than clicked together by hand, the application runs on a managed container orchestrator, and the whole system is observable once it is live.

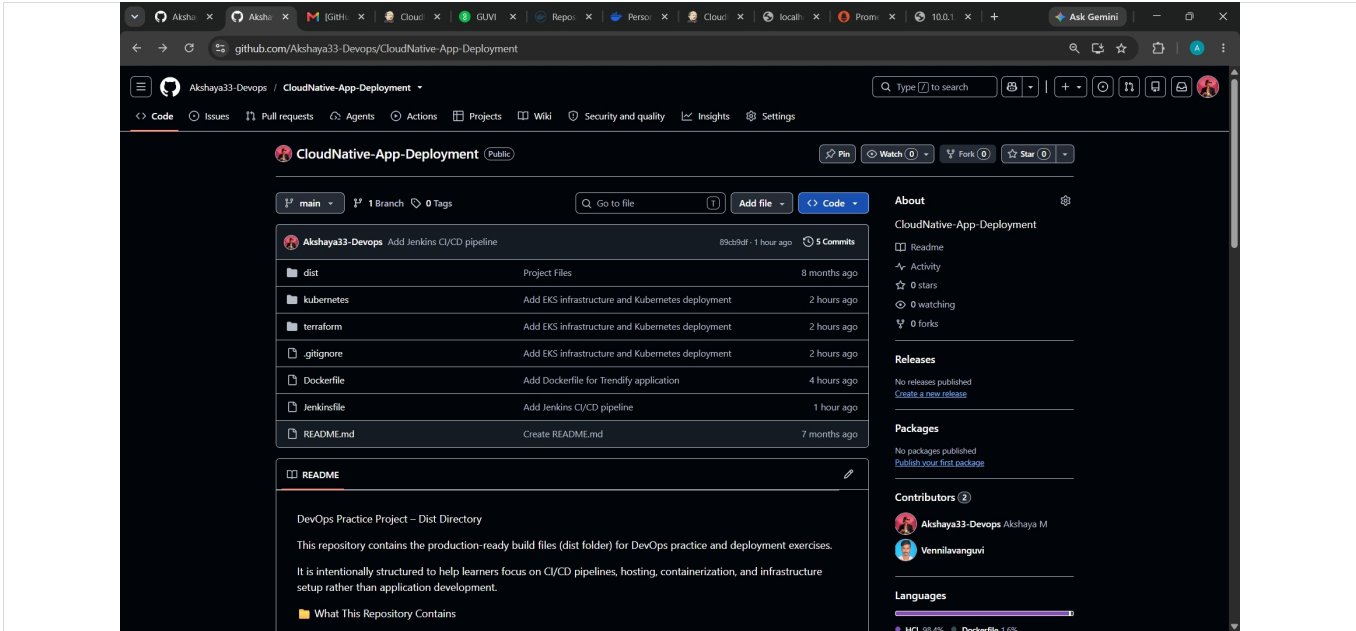


Figure 1.1 — GitHub repository: Akshaya33-Devops/CloudNative-App-Deployment

1.2 Why this specific stack

Each tool was chosen to demonstrate a distinct layer of the DevOps lifecycle, and each one replaces a manual, error-prone step:

Layer	Tool	Problem it solves
Source control	GitHub	Single source of truth for application code, manifests, and pipeline definitions; enables collaboration and rollback.
Containerization	Docker	Packages the app with its runtime (Nginx) so it behaves identically on a laptop, in Jenkins, and inside EKS.
Image registry	Docker Hub	Central, versioned storage for built images so Kubernetes can pull the exact artifact Jenkins produced.
Infrastructure as Code	Terraform	Makes infrastructure reproducible, reviewable, and destroyable — no manual console clicking that can't be audited or repeated.
Orchestration	Amazon EKS / Kubernetes	Runs multiple replicas, restarts failed pods, and abstracts away individual servers.

Layer	Tool	Problem it solves
CI/CD	Jenkins	Removes humans from the build-test-deploy loop; every push can result in a deployment without manual steps.
Networking	AWS Load Balancer / VPC	Exposes the app to the internet securely and distributes traffic across pods.
Observability	Prometheus, Grafana, Metrics Server	Turns an opaque running cluster into something you can see: resource usage, health, and alerts.

1.3 Why this matters beyond “it works”

A deployment that only works once, by hand, teaches nothing about production engineering. This project is deliberately built so that:

- Anyone can rebuild the entire environment from the Git repository alone (Terraform + Jenkinsfile + manifests), with no hidden manual steps.
- Failures are recoverable: because infrastructure is code, a broken environment can be destroyed and re-created rather than debugged by hand.
- The system reports its own health, so a failure is visible in Grafana/Prometheus before a user reports it.
- Access is controlled through IAM roles rather than long-lived secrets, reducing the blast radius of a leaked credential.

2. Architecture — Explaining Every Flow

The system has three parallel flows that share the same EKS cluster: the CI/CD flow (how code becomes a running pod), the request flow (how a user's browser reaches that pod), and the monitoring flow (how cluster health data reaches a dashboard). Understanding them separately makes the whole system easier to reason about and to troubleshoot.

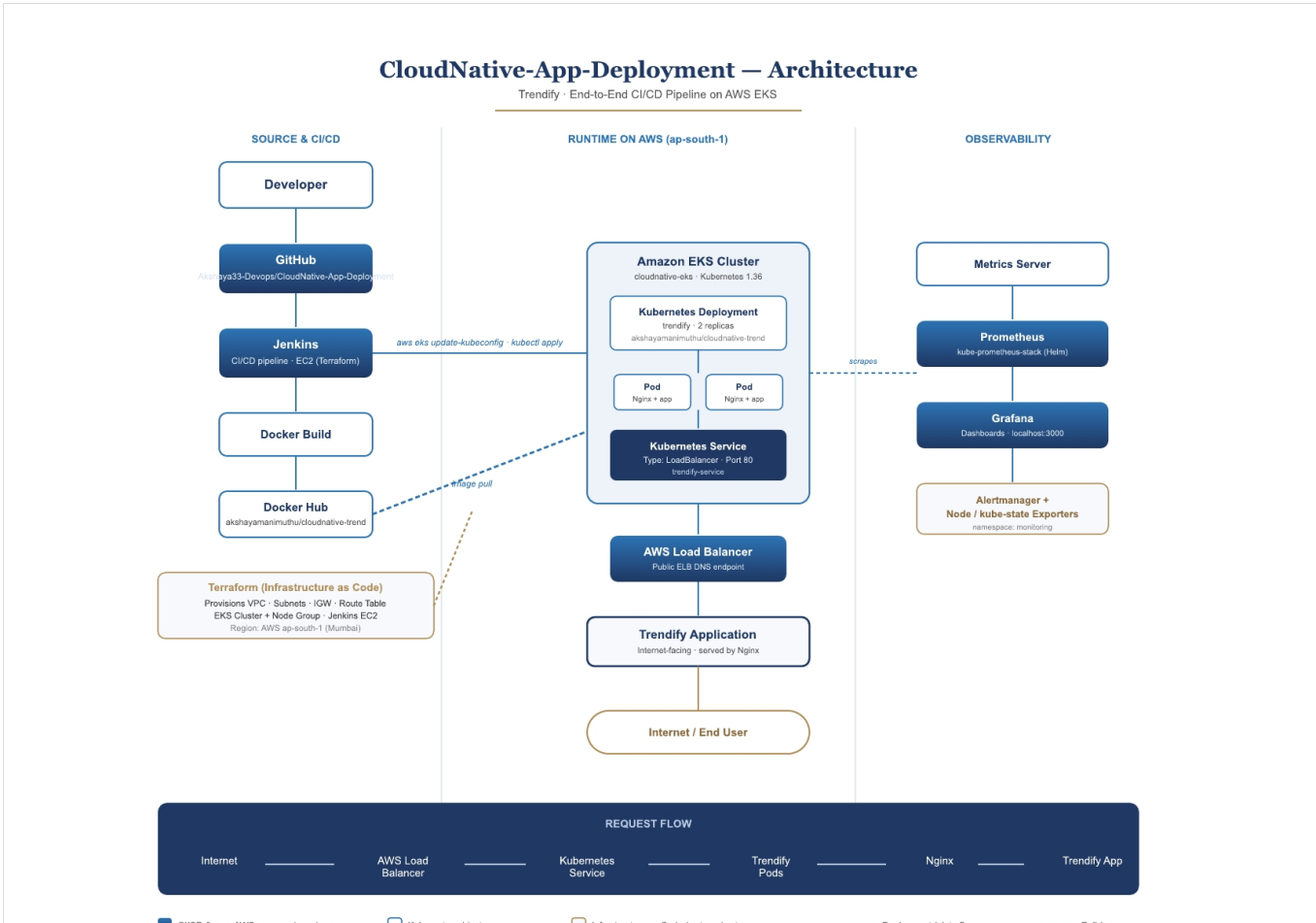


Figure 2.1 — CloudNative-App-Deployment full architecture diagram (source & CI/CD, runtime on AWS, observability)

2.1 CI/CD flow — how code becomes a running container

```

Developer
|  git push
v
GitHub (source of truth: app code, Dockerfile, Jenkinsfile, Terraform, manifests)
|  webhook / poll triggers a build
v
Jenkins (running on an EC2 instance provisioned by Terraform)
|
+--> Docker Build    (packages dist/ + Nginx into an image)
|
+--> Docker Push     (publishes the image to Docker Hub, tagged :BUILD_NUMBER and :dev)
|
v
Amazon EKS
|
+--> kubectl apply / kubectl set image (rolls the new image out to the Deployment)
|
v
Trendify Pods (new ReplicaSet created, old pods terminated once new ones are Ready)

```

Why it's built this way: Jenkins never pushes to Kubernetes using a developer's local credentials. It authenticates to AWS using an IAM role attached to its own EC2 instance, and it is separately granted permission inside EKS via an access entry. This means the pipeline can run unattended and the blast radius of a compromised Jenkins box is limited to exactly the permissions that role was given — nothing more.

2.2 Request flow — how a user reaches the application

```

Internet
|
v
AWS Load Balancer (public ELB DNS name, created automatically because the
                  Kubernetes Service type is LoadBalancer)
|
v
Kubernetes Service (trendify-service) — a stable virtual IP + DNS name that
                  load-balances across all healthy pod replicas
|
v
Trendify Pods (2 replicas, each running Nginx)
|
v
Nginx — serves the pre-built Vite dist/ static files
|
v
Trendify Application (rendered in the browser)

```

Why a Service sits in front of the Pods: Pods are ephemeral — Kubernetes can kill and reschedule them at any time, and each new Pod gets a new internal IP. The Service gives clients (in this case, the AWS Load Balancer) one stable address to talk to, and Kubernetes keeps the list of “healthy pod IPs behind that address” up to date automatically via label selectors and readiness checks.

2.3 Monitoring flow — how health data reaches a dashboard

```

EKS Cluster (nodes, pods, containers)
|
+--> Metrics Server  (lightweight, in-cluster resource metrics: CPU/memory)
    |      used by:  kubectl top nodes / kubectl top pods
    |
+--> Node Exporter + kube-state-metrics  (expose detailed metrics as HTTP endpoints)
    |
    v
    Prometheus  (scrapes those endpoints on an interval, stores time-series data)
    |
    v
    Grafana  (queries Prometheus, renders dashboards)
    |
    Alertmanager  (fires alerts when Prometheus rules are breached)
  
```

Why two separate metrics systems exist: Metrics Server is intentionally minimal — it exists only to answer “how much CPU/memory is this pod using right now,” which the Kubernetes scheduler and `kubectl top` rely on. It keeps no history. Prometheus is the opposite: it stores a time series of everything it scrapes, which is what makes trend graphs, alerting rules, and historical comparisons in Grafana possible. Running both is standard practice — one is for the control plane, the other is for humans and dashboards.

3. Tech Stack

Category	Tools / Services
Source Control	GitHub
CI/CD	Jenkins
Containerization	Docker, Docker Hub
Infrastructure as Code	Terraform
Orchestration	Amazon EKS, Kubernetes
Networking	AWS Load Balancer, VPC, Subnets, Internet Gateway
Monitoring	Prometheus, Grafana, Metrics Server, Helm
Cloud Provider	AWS (region: ap-south-1)

4. Project Structure

```

Trend/
├── Jenkinsfile
├── Dockerfile
├── dist/
├── kubernetes/
│   ├── deployment.yaml
│   └── service.yaml
├── terraform/
│   ├── eks.tf
│   ├── jenkins.tf
│   ├── eks_iam.tf
│   ├── node_group.tf
│   ├── provider.tf
│   ├── vpc.tf
│   ├── subnets.tf
│   ├── route_table.tf
│   └── internet_gateway.tf
├── .gitignore
└── README.md

```

Keeping Terraform files split by concern (vpc.tf, subnets.tf, eks.tf, node_group.tf, jenkins.tf, eks_iam.tf) rather than one giant main.tf makes the codebase easier to review — each file answers one question (“where does networking live,” “what does the cluster look like,” “what can Jenkins do”) and merge conflicts are less likely when multiple people touch infrastructure at once.

5. Prerequisites

Before running any part of this project, the following need to be installed and configured locally or on the provisioning machine:

Tool	Purpose	Verify with
AWS CLI (configured)	Authenticates Terraform and kubectl to AWS	<code>aws sts get-caller-identity</code>
Terraform	Provisions VPC, EKS, node group, Jenkins EC2	<code>terraform version</code>
Docker	Builds and runs the container image locally	<code>docker version</code>
kubectl	Talks to the EKS control plane	<code>kubectl version --client</code>
Helm	Installs the kube-prometheus-stack chart	<code>helm version</code>
Git	Clones the repository and drives the Jenkins pipeline	<code>git --version</code>

An AWS account with permission to create VPCs, EC2 instances, IAM roles, and EKS clusters is also required, along with a Docker Hub account for the image registry.

6. Setup & Deployment Guide

6.1 Clone the Repository

```
git clone https://github.com/Akshaya33-Devops/CloudNative-App-Deployment.git
cd CloudNative-App-Deployment/Trend
```

Why `cd` into `Trend/`: the repository root holds project-level metadata, but the `Jenkinsfile`, `Dockerfile`, and manifests all assume the `Trend/` directory as the working directory — Docker build context, `kubectl` manifest paths, and Terraform's relative paths are all resolved from here.

6.2 Containerize the Application — Docker

The Trendify app ships as a production-ready Vite build (`dist/`) — already transpiled, minified, static HTML/CSS/JS. It does not need Node.js at runtime, so the container only needs a web server to serve static files.

Dockerfile

```
FROM nginx:alpine

COPY dist/ /usr/share/nginx/html/

EXPOSE 80
```

Why each line is there:

- **FROM nginx:alpine** — `nginx:alpine` is used instead of a full `nginx` image or a Node-based image because the app is pre-built static assets. Alpine keeps the base image small (tens of MB instead of hundreds), which means faster pulls in Jenkins and in EKS, and a smaller attack surface since Alpine ships far fewer packages than a general-purpose distro.
- **COPY dist/ /usr/share/nginx/html/** — copies the already-built static site into Nginx's default web root. No build step happens inside the image — build once, ship the same artifact everywhere — which is why the pipeline doesn't run `npm install/npm run build` inside Docker.
- **EXPOSE 80** — documents that the container listens on port 80 (Nginx's default). This is metadata for readers and orchestrators; it doesn't publish the port by itself — that happens at `docker run` or in the Kubernetes Service.

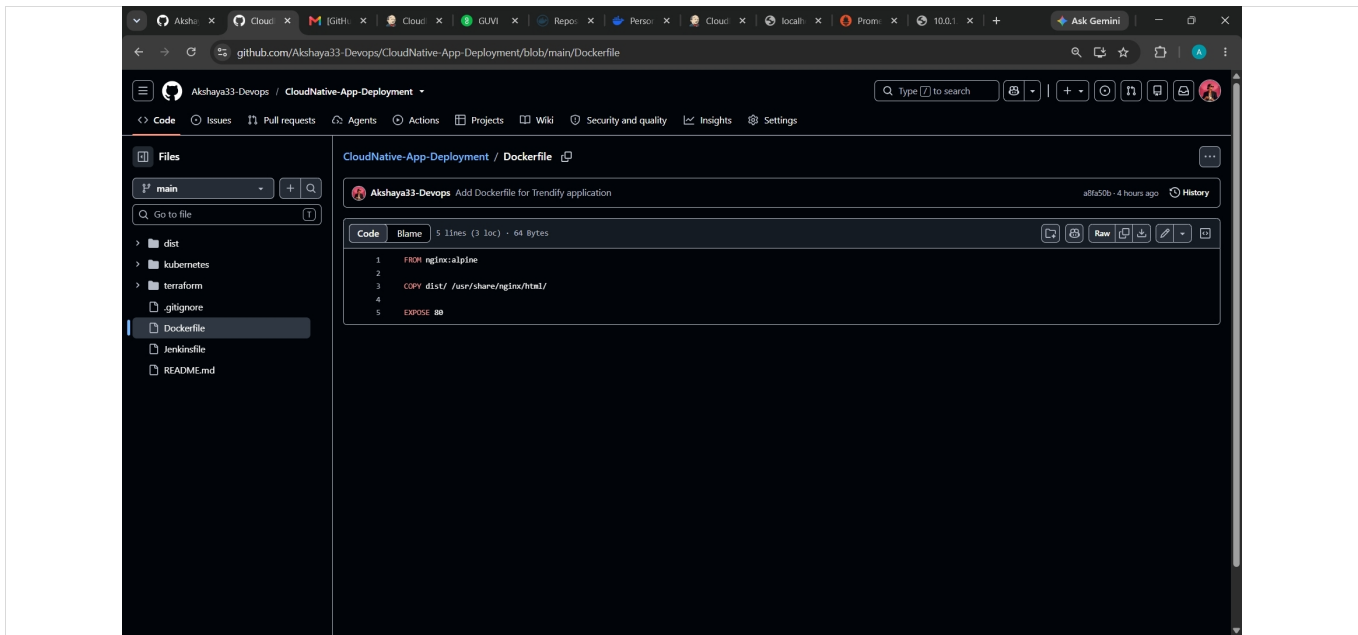


Figure 6.1 — Dockerfile contents in the GitHub repository

Build and run locally

```
docker build -t akshayanimuthu/cloudnative-trend:dev .
docker run -d -p 8081:80 akshayanimuthu/cloudnative-trend:dev
```

Why each flag matters:

- **docker build -t** — the -t tag follows the <dockerhub-username>/<repo>:<tag> convention so the same name can be pushed straight to Docker Hub later without renaming. The trailing . sets the build context to the current directory (Trend/), which is why the command must be run from there.
- **docker run -d -p 8081:80 ...** — -d runs the container in the background (detached); -p 8081:80 maps host port 8081 to the container's port 80 so the app can be checked at <http://localhost:8081> without colliding with anything else already using port 80/8080 on the host. In production this mapping doesn't exist — Kubernetes talks to port 80 inside the pod network directly.

Verify: open <http://localhost:8081> and confirm the Trendify UI renders. In production the image is never pushed by hand — the Jenkins pipeline builds and pushes it automatically on every run, tagged both :dev and :<jenkins-build-number> so a specific build can always be traced back to a specific Jenkins run.

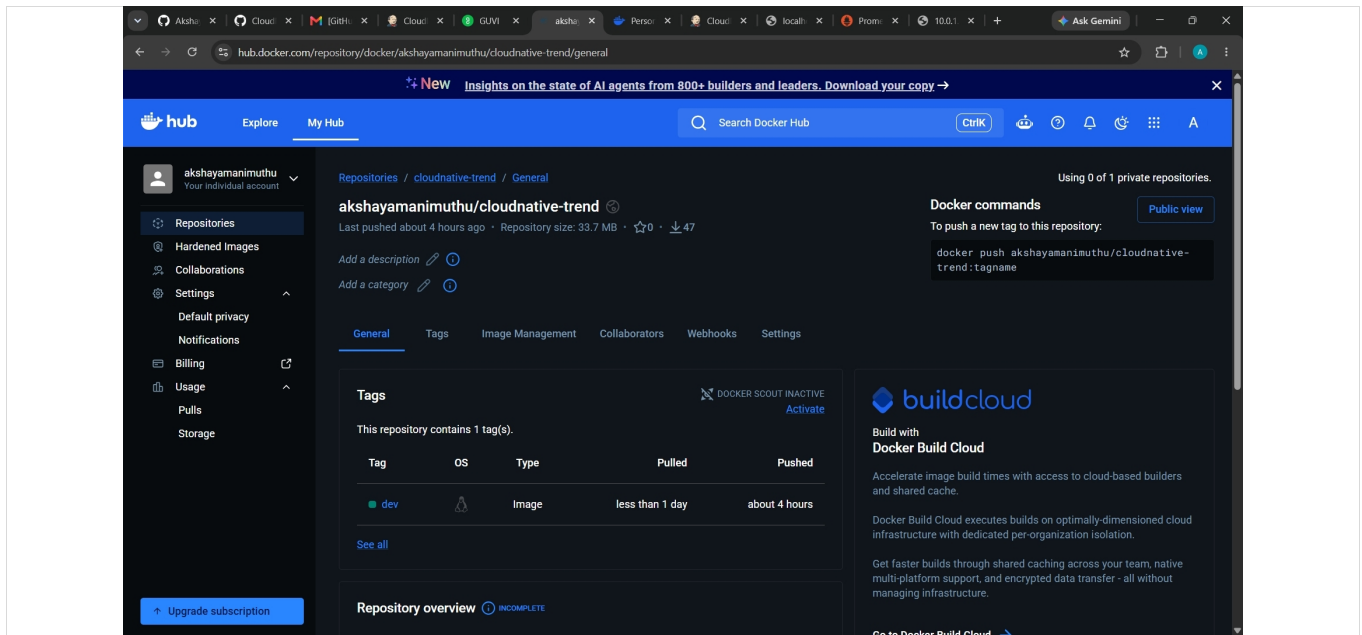


Figure 6.2 — `akshayanimuthu/cloudnative-trend` repository on Docker Hub

6.3 Provision AWS Infrastructure with Terraform

Terraform provisions the entire networking and compute stack in ap-south-1 before any application code runs. Doing this as code (rather than clicking through the AWS console) means the environment can be torn down and rebuilt identically, reviewed like any other code change, and destroyed completely when not needed — important for avoiding ongoing AWS charges on a portfolio project.

```
VPC (10.0.0.0/16)
├── Public Subnet 1 (10.0.1.0/24)
├── Public Subnet 2 (10.0.2.0/24)
├── Internet Gateway
├── Route Table
├── EKS Cluster
├── EKS Node Group (t3.small, min 1 / desired 2 / max 3)
└── Jenkins EC2 instance
```

What each Terraform-managed component is for

File / Resource	What it creates	Why it's needed
vpc.tf	A VPC (10.0.0.0/16)	An isolated private network boundary for everything else — nothing here shares a network with other AWS accounts or the public internet by default.
subnets.tf	Two public subnets across two Availability Zones	EKS requires subnets in at least two AZs for high availability; “public” means resources in them can get a route to the internet via the gateway.
internet_gateway.tf	An Internet Gateway attached to the VPC	Without it, nothing inside the VPC — including the Load Balancer — could reach or be reached from the internet.
route_table.tf	A route table sending 0.0.0.0/0 traffic to the Internet Gateway	Defines the actual path traffic takes to leave the VPC; the subnets are only “public” because they're associated with this route table.
eks.tf	The EKS control plane (cloudnative-eks)	The managed Kubernetes API server and scheduler; AWS operates and patches this, so the project doesn't have to run its own control plane.
node_group.tf	A managed node group (t3.small, min 1 / desired 2 / max 3)	The actual EC2 worker nodes that run pods. t3.small keeps cost low for a demo workload; the min/desired/max range lets the group self-heal and, if configured with autoscaling, react to load.
eks_iam.tf	IAM roles for the EKS cluster and node group, plus the Jenkins access entry	EKS requires specific AWS-managed policies attached to its cluster role and node role to function at all — this is also where Jenkins's EKS permissions are defined (see 6.4 and Section 7).
jenkins.tf	An EC2 instance pre-installed with Jenkins, Git, Docker, AWS CLI, kubectl, plus its security group and IAM role	Gives the pipeline a dedicated, disposable build server instead of running Jenkins on a developer machine or inside the cluster it's deploying to.

File / Resource	What it creates	Why it's needed
provider.tf	The AWS provider configuration (region, etc.)	Tells Terraform which cloud and region to talk to; keeping it separate makes the region a single, obvious place to change.

Running it

```
cd terraform
terraform init
terraform plan
terraform apply
```

Why each step is separate rather than just terraform apply -auto-approve:

- **terraform init** — downloads the AWS provider plugin and sets up the local backend/state. This only needs to happen once per checkout (or when providers/modules change).
- **terraform plan** — shows exactly what will be created, changed, or destroyed before anything actually happens. Reading the plan is the single most important habit in Infrastructure as Code — it's the difference between an intentional change and an accidental one (e.g. a plan that shows a VPC being destroyed and recreated should stop you immediately).
- **terraform apply** — actually creates the resources, but still asks for a typed “yes” confirmation by default — a deliberate manual checkpoint before real AWS resources (and billing) are created.

The EKS cluster is created with authentication mode `API_AND_CONFIG_MAP`, which allows cluster access to be granted through both the newer IAM access-entry API and the legacy aws-auth ConfigMap. This project uses the access-entry approach specifically so Jenkins can be granted cluster access without editing a ConfigMap by hand.

```

region = "ap-south-1"
revoke_rules_on_delete = false
tags = {
  "Name" = "cloudnative-jenkins-sg"
}
tags_all = {
  "Name" = "cloudnative-jenkins-sg"
}
vpc_id = "vpc-0f0a0f679c3d4c49d"
}

Plan: 6 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_role.jenkins_role: Creating...
aws_security_group.jenkins: Creating...
aws_iam_role.jenkins_role: Creation complete after 3s [id=cloudnative-jenkins-role]
aws_iam_role_policy_attachment.jenkins_ec2: Creating...
aws_iam_instance_profile.jenkins_profile: Creating...
aws_iam_role_policy_attachment.jenkins_eks: Creating...
aws_security_group.jenkins: Creation complete after 3s [id=sg-009ea7b09c440c970]
aws_iam_role_policy_attachment.jenkins_ec2: Creation complete after 1s [id=cloudnative-jenkins-role/arn:aws:iam::aws:policy/AmazonEC2FullAccess]
aws_iam_role_policy_attachment.jenkins_eks: Creation complete after 1s [id=cloudnative-jenkins-role/arn:aws:iam::aws:policy/AmazonEKSClusterPolicy]
aws_iam_instance_profile.jenkins_profile: Creation complete after 8s [id=cloudnative-jenkins-profile]
aws_instance.jenkins: Creating...
aws_instance.jenkins: Still creating... [00m10s elapsed]
aws_instance.jenkins: Creation complete after 14s [id=i-0a45c408569f78c88]

Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
PS D:\guvi\CloudNative-App-Deployment\Trend\terraform> aws ec2 describe-instances --region ap-south-1 --instance-ids i-0a45c408569f78c88 --query "Reservations[0].Instances[0].{State:State,Name:PublicIP:PublicIpAddress,PrivateIP:PrivateIpAddress}" --output table

```

PrivateIP	PublicIP	State
10.0.1.121	13.232.208.215	running

Figure 6.3 — Terraform apply provisioning the Jenkins IAM role, security group and EC2 instance

6.4 Configure Jenkins

Terraform provisions a Jenkins EC2 instance pre-installed with Jenkins, Git, Docker, AWS CLI, and kubectl, so no manual software installation is needed after terraform apply finishes.

How Jenkins authenticates to AWS

Jenkins authenticates to AWS via an IAM role attached to the EC2 instance itself (cloudnative-jenkins-role) — not static AWS access keys stored as Jenkins credentials. This matters for two reasons:

- No long-lived secret exists that could leak from Jenkins configuration, logs, or a compromised plugin.
- The AWS SDK/CLI on the instance automatically picks up temporary, auto-rotating credentials from the EC2 instance metadata service — nothing to configure or renew by hand.

EKS access for this role is granted using an `aws_eks_access_entry` bound to the `AmazonEKSAAdminPolicy`, which is what actually lets Jenkins run `kubectl` commands against the cluster (see Section 7 for the full explanation of this handshake).

Docker Hub credentials

Jenkins stores a single global credential (ID: `dockerhub-creds`) containing the Docker Hub username and an access token — not the account password — used only during the Docker Push pipeline stage. Using a scoped access token instead of the real password means it can be revoked independently without changing the Docker Hub login itself.

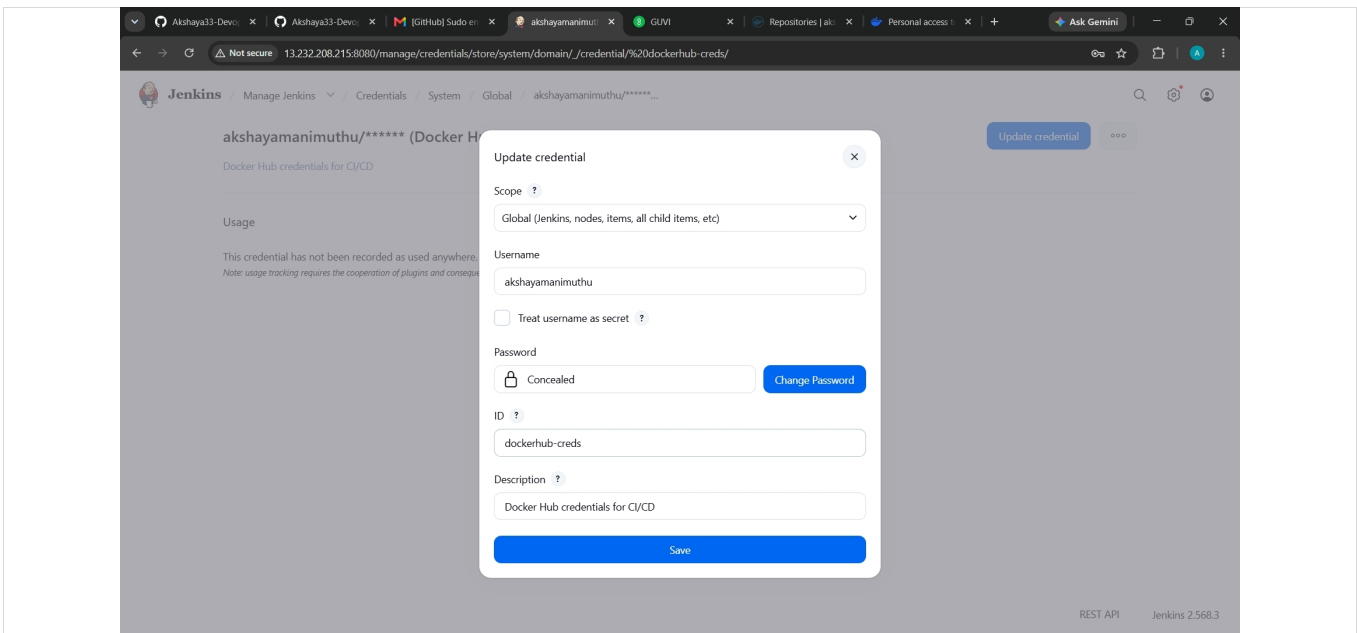


Figure 6.4 — Jenkins global credential (`dockerhub-creds`) used for Docker Hub authentication

6.5 Deploy to Amazon EKS — Kubernetes Objects

Deployment (kubernetes/deployment.yaml)

- 2 replicas
- Image: akshayamanimuthu/cloudnative-trend

A Deployment is a controller object, not the pods themselves — it describes the desired state (“2 copies of this image should always be running”) and Kubernetes continuously reconciles reality toward that state. If a pod crashes, is evicted, or its node fails, the Deployment's controller notices the replica count has dropped and schedules a replacement automatically, with no human involvement. Running 2 replicas rather than 1 also means the app survives a single pod restart (e.g. during a rolling update) without downtime.

Service (kubernetes/service.yaml)

- Type: LoadBalancer
- Port: 80 → Target Port: 80

Setting the Service type to LoadBalancer is what triggers the AWS cloud-controller-manager to provision a real AWS Elastic Load Balancer automatically and wire its target group to the pods matching this Service's selector. This is the key integration point between Kubernetes' abstract networking model and AWS's actual infrastructure — the same YAML would provision a different kind of load balancer on GCP or Azure, which is part of Kubernetes' appeal as a portable deployment target.

Applying manually (outside the pipeline)

```
aws eks update-kubeconfig --region ap-south-1 --name cloudnative-eks
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
```

update-kubeconfig writes (or updates) an entry in ~/.kube/config with the cluster's API endpoint and a way to authenticate to it, so every subsequent kubectl command in the session targets this cluster specifically. kubectl apply is declarative and idempotent — running it again with an unchanged file does nothing, and running it with a changed file (e.g. a new image tag) triggers exactly the diff needed, which is why the same command is safe to use both for the very first deploy and for every update after it.

In normal operation, these three commands are run by Jenkins, not by hand — manual application is only for initial bootstrap or recovery.

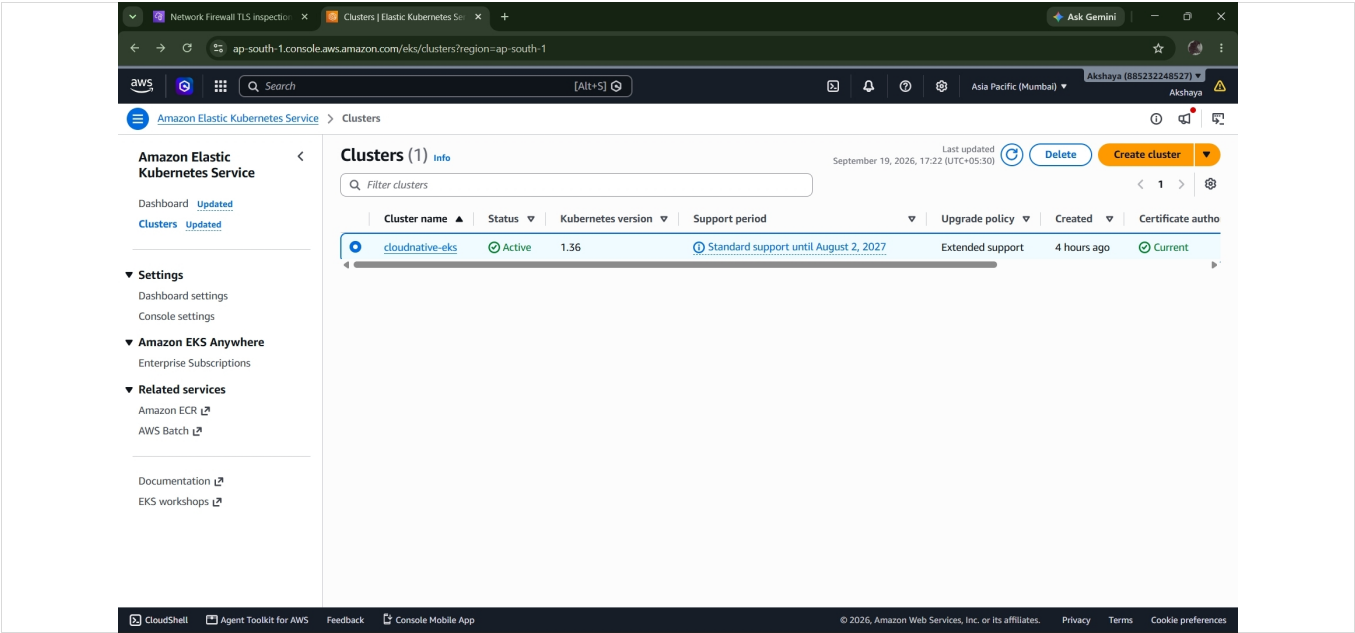


Figure 6.5 — cloudnative-eks cluster, Active, Kubernetes version 1.36 (Amazon EKS console)

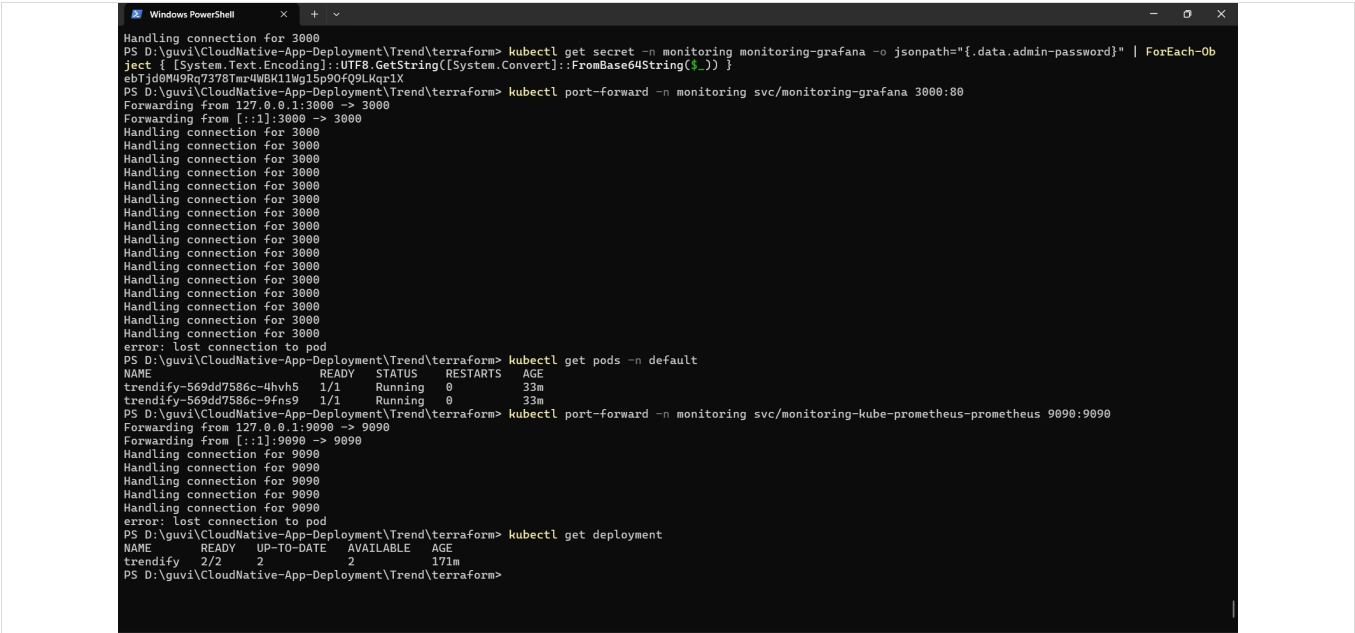


Figure 6.6 — kubectl get deployment confirming trendify 2/2 replicas available

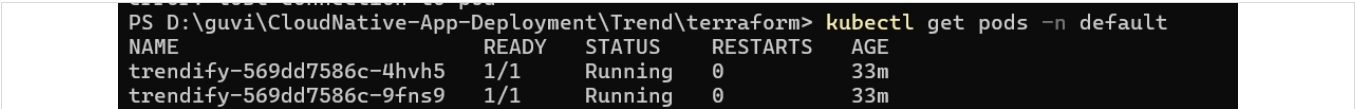


Figure 6.7 — kubectl get pods -n default: Trendify pods Running

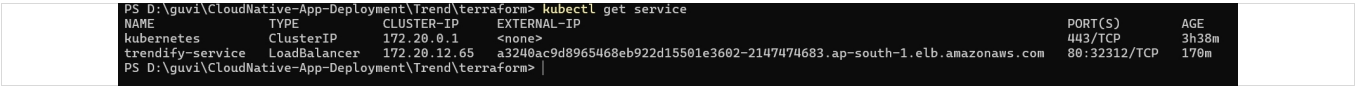


Figure 6.8 — kubectl get service: trendify-service exposed via LoadBalancer

6.6 CI/CD Pipeline — Jenkins Stage by Stage

The Jenkinsfile defines a declarative pipeline with the following stages, run in order on every build:

<pre> Checkout v Docker Build v Docker Push v Configure EKS v Deploy to EKS v Verify Deployment </pre>		
Stage	What it does	Why it exists
Checkout	Pulls the main branch from GitHub	Guarantees every build starts from a known, versioned commit — not whatever happens to be on disk.
Docker Build	Builds the image, tagged <code>:\${BUILD_NUMBER}</code> and <code>:dev</code>	Two tags serve two purposes: the build-number tag gives every image a unique, traceable identity tied to the exact Jenkins run; <code>:dev</code> is a floating tag for quick local/manual testing.
Docker Push	Pushes both tags to Docker Hub using <code>dockerhub-creds</code>	Kubernetes pulls images from a registry, not from the Jenkins filesystem — the image has to leave the build machine before EKS can use it.
Configure EKS	Runs <code>aws eks update-kubeconfig</code> to point <code>kubectl</code> at the cluster	Jenkins's own <code>kubeconfig</code> has to be set on every run (its filesystem may not persist between builds), so this step can't be skipped even though it looks repetitive.
Deploy to EKS	Applies manifests and updates the Deployment's image via <code>kubectl set image</code>	<code>kubectl set image</code> is what actually triggers a rolling update — it changes only the image reference on the running Deployment rather than reapplying the whole manifest, which is a smaller, more surgical change.
Verify Deployment	Runs <code>kubectl rollout status</code> , <code>kubectl get pods</code> , and <code>kubectl get service</code>	Turns “the apply command didn't error” into “the new pods are actually Running and the Service still has an external address” — a real deployment gate, not just a fire-and-forget apply.

Key verification commands run by the pipeline

```
kubectl rollout status deployment/trendify
```

```
kubectl get pods
kubectl get service trendify-service
```

kubectl rollout status blocks and returns a non-zero exit code if the rollout doesn't complete — which is what allows the pipeline stage to genuinely fail (and alert whoever is watching) instead of reporting success just because the apply command was accepted. A successful pipeline run ends with the trendify Deployment reporting 2/2 pods Running and Jenkins printing “Trendify deployment completed successfully” / Finished: SUCCESS.

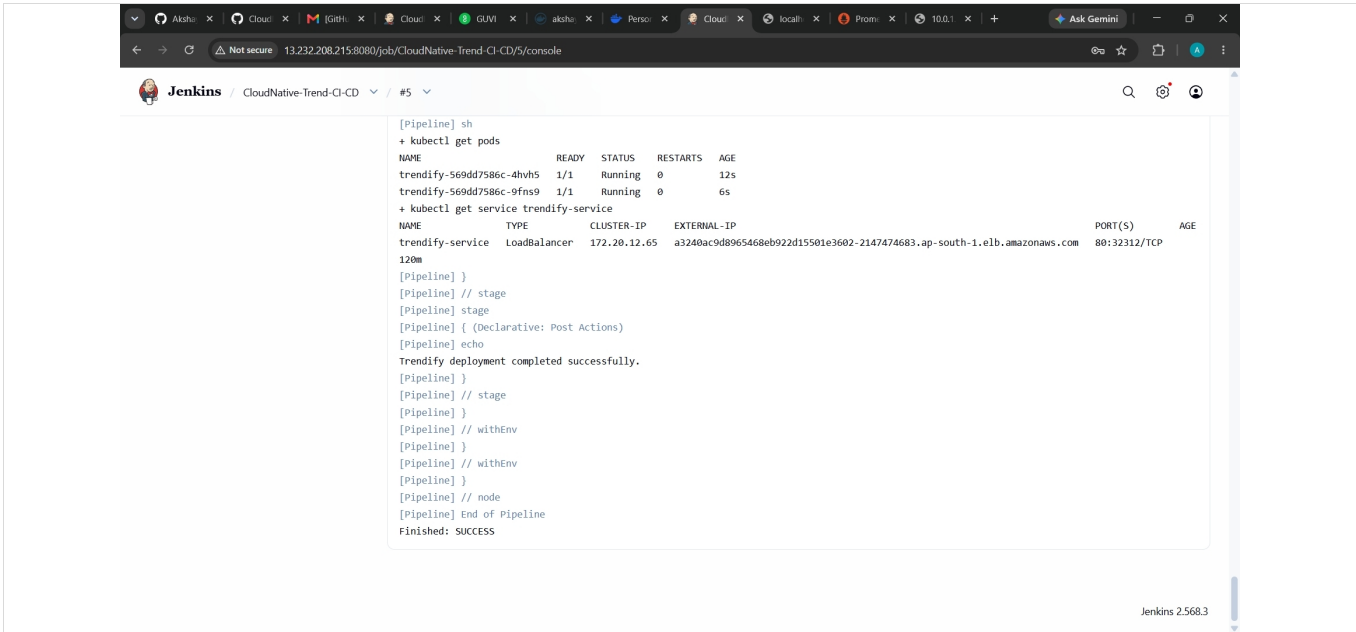


Figure 6.9 — Jenkins pipeline console output: “Trendify deployment completed successfully” / Finished: SUCCESS

Why the pipeline is written as a declarative Jenkinsfile

Storing the pipeline definition (Jenkinsfile) in the same Git repository as the application code means the deployment process is versioned alongside the code it deploys — a change to the pipeline goes through the same review and history as any other code change, and Jenkins can reconstruct exactly how a given commit was built and shipped months later.

6.7 Monitoring — Helm, Prometheus, Grafana, Metrics Server

Installing the stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Helm is the package manager for Kubernetes — it installs a “chart” (a bundle of pre-written, parameterized Kubernetes manifests) instead of hand-writing dozens of YAML files for Prometheus, Grafana, Alertmanager, and their supporting RBAC, ServiceAccounts, and ServiceMonitors.

```
kubectl create namespace monitoring
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

A dedicated monitoring namespace isolates the observability stack's resources (and their RBAC) from the application namespace — they can be listed, upgraded, or deleted as one unit without touching the trendify Deployment.

What each component actually does

Component	Role
Prometheus	Pulls (“scrapes”) metrics from targets on a schedule and stores them as time series; also evaluates alerting rules.
Grafana	Queries Prometheus's stored data and renders it as dashboards — CPU/memory over time, per-namespace usage, pod counts, and more.
Alertmanager	Receives firing alerts from Prometheus and routes them (e.g. to email/Slack) with deduplication and grouping — not covered in depth in this deployment but installed as part of the stack.
Prometheus Operator	A Kubernetes controller that lets Prometheus be configured declaratively via CRDs (like ServiceMonitor) instead of hand-edited config files.
Node Exporter	Runs on every node and exposes host-level metrics (CPU, memory, disk, network) that Prometheus scrapes.
kube-state-metrics	Exposes the state of Kubernetes objects themselves (deployment replica counts, pod phase, etc.) as metrics — different from resource usage, this is object state.
Metrics Server	A separate, minimal install that provides live CPU/memory numbers used by kubectl top and by the Horizontal Pod Autoscaler; it keeps no history.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
kubectl top nodes
```

Why both Prometheus and Metrics Server run side by side: Metrics Server answers a narrow, latency-sensitive question (“how much CPU is this pod using right now”) for the scheduler and autoscaler, with no storage overhead. Prometheus answers a broader question (“what has resource usage looked like over the last hour, and should anyone be alerted”) at the cost of storing history. Neither replaces the other.

```
PS D:\guvi\CloudNative-App-Deployment\Trend\terraform> kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0  2/2     Running   0           38m
monitoring-grafana-7768b5dffb-ltnsr  3/3     Running   0           38m
monitoring-kube-prometheus-operator-79bcc95ddd-r7tm2    1/1     Running   0           38m
monitoring-kube-state-metrics-764584797b-hnvz5         1/1     Running   0           38m
monitoring-prometheus-node-exporter-8v7cr              1/1     Running   0           38m
monitoring-prometheus-node-exporter-rv28v              1/1     Running   0           38m
prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running   0           38m
PS D:\guvi\CloudNative-App-Deployment\Trend\terraform> |
```

Figure 6.10 — `kubectl get pods -n monitoring`: Prometheus, Grafana, Alertmanager, and exporter pods Running

Accessing the dashboards

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

`port-forward` is used here instead of another LoadBalancer Service because exposing Grafana and Prometheus directly to the internet would need its own authentication hardening; for this project's scope, tunneling through `kubectl` — which already requires valid cluster credentials — is the simpler, safer choice. Grafana is reached at `http://localhost:3000`, logging in with the admin credentials stored in the Kubernetes Secret created by the Helm chart.

```
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090
```

Prometheus's own UI at `http://localhost:9090` (Status → Target health) is the fastest way to confirm every ServiceMonitor is actually being scraped — an UP target confirms the entire chain (exporter running → Service reachable → Prometheus configured correctly) in one screen.

7. How Jenkins Connects to EKS

This is one of the parts most worth being able to explain clearly in an interview, because it's the security-critical handshake between the CI/CD layer and the orchestration layer.

1. Identity: the Jenkins EC2 instance is launched with an IAM instance profile attached, which wraps the IAM role `cloudnative-jenkins-role`. Anything running on that instance — including Jenkins pipeline steps — can request temporary AWS credentials for that role from the EC2 instance metadata service, with no keys ever stored on disk.
2. AWS-level permission: that role has an IAM policy (`AmazonEC2FullActions`/relevant EKS permissions in this project) letting it call the EKS API, including `eks:DescribeCluster` and `eks:UpdateKubeconfig`-related calls.
3. Getting a kubeconfig: `aws eks update-kubeconfig --region ap-south-1 --name cloudnative-eks` asks AWS for the cluster's API endpoint and certificate, and configures `kubectl` to authenticate every request using the instance's IAM identity (via the AWS IAM Authenticator mechanism built into `kubectl`/AWS CLI) — not a separate Kubernetes-native credential.
4. Kubernetes-level permission: having valid AWS credentials only proves who you are to EKS — it does not by itself grant any permission inside the Kubernetes API (RBAC). This project's EKS cluster uses authentication mode `API_AND_CONFIG_MAP`, and an `aws_eks_access_entry` maps the `cloudnative-jenkins-role` IAM role to the `AmazonEKSAAdminPolicy` access policy. That access entry is the actual authorization step — without it, Jenkins could authenticate to the cluster but every `kubectl` command would fail with `Forbidden`.
5. Running commands: once both the AWS-level and Kubernetes-level permissions line up, `kubectl apply`, `kubectl set image`, and `kubectl rollout status` in the pipeline work exactly as they would from a human operator's laptop.

The takeaway for an interview: EKS authentication and authorization are two separate systems layered on top of each other — IAM decides who you are, and a Kubernetes-side mapping (access entry or `aws-auth` ConfigMap) decides what that identity is allowed to do once inside the cluster. A common real-world bug is granting the IAM side but forgetting the access-entry/RBAC side (or vice versa), which is exactly what Troubleshooting item 10.4.4 covers.

8. Accessing the Application

Because the Service type is LoadBalancer, Kubernetes provisions an AWS Elastic Load Balancer automatically and the app becomes publicly reachable at the generated ELB DNS name, for example:

```
http://a3240ac9d8965468eb922d15501e3602-2147474683.ap-south-1.elb.amazonaws.com
```

Request flow (see Section 2.2 for the detailed explanation):

```
Internet -> AWS Load Balancer -> Kubernetes Service -> Trendify Pods -> Nginx -> Trendify Application
```

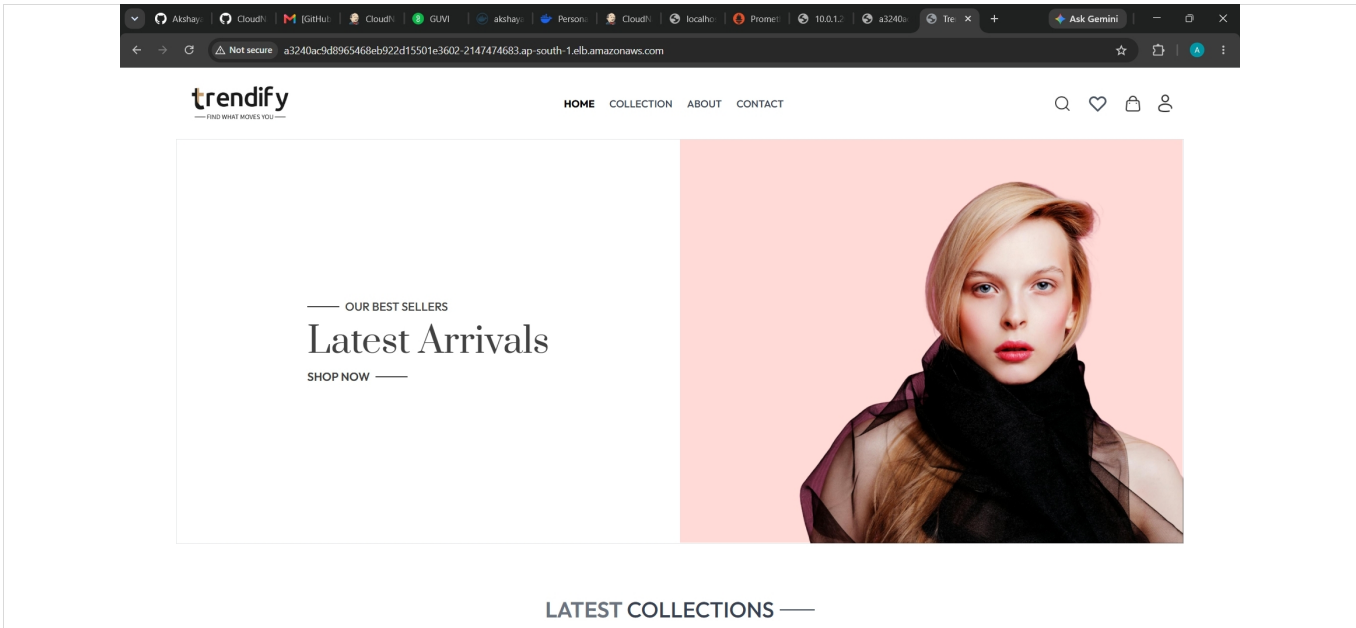


Figure 7.1 — Trendify application live on the AWS Load Balancer endpoint

9. Accessing Monitoring Dashboards

9.1 Grafana

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

Open <http://localhost:3000> and log in with the admin credentials stored in the Kubernetes Secret. The built-in Kubernetes dashboards (Compute Resources / Cluster, Namespace, Pod, Workload, Node) cover CPU/memory utilization, requests vs. limits, and per-namespace resource quotas without writing a single query by hand.

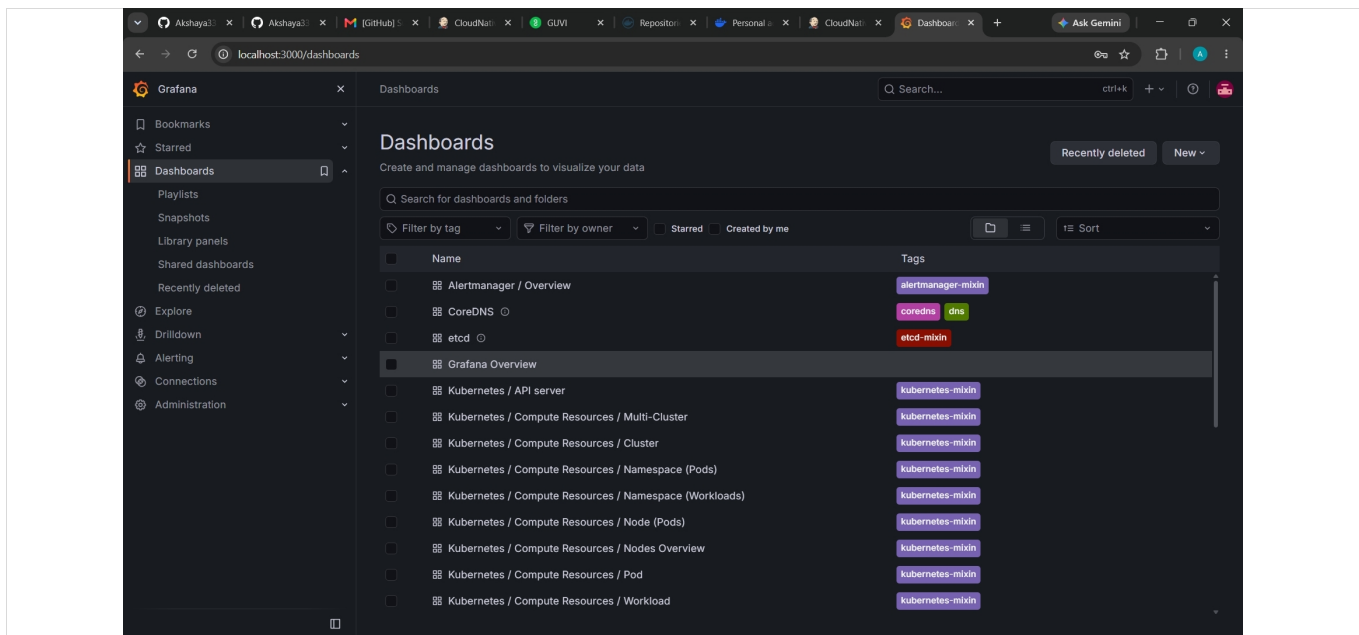


Figure 8.1 — Grafana dashboard list, including the Kubernetes / Compute Resources dashboard set

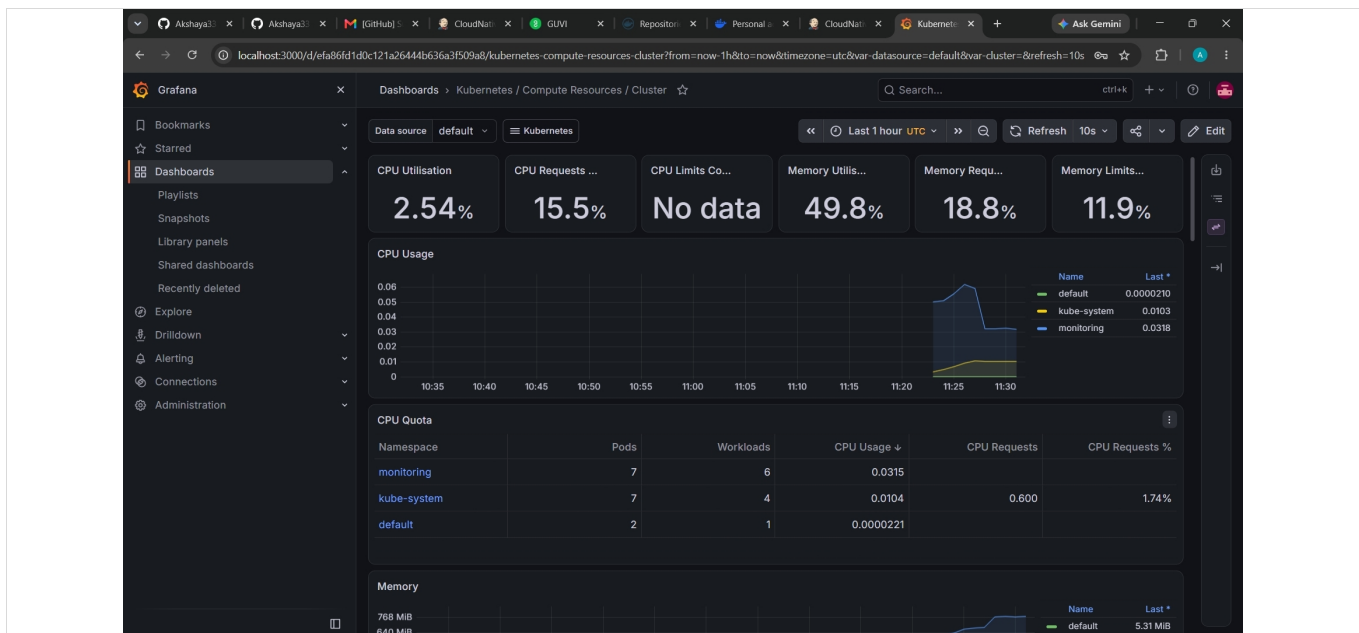


Figure 8.2 — Grafana "Kubernetes / Compute Resources / Cluster" dashboard showing CPU and memory utilisation

9.2 Prometheus

```
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090
```

Open <http://localhost:9090> and check Status → Target health to confirm every scrape target reports UP.

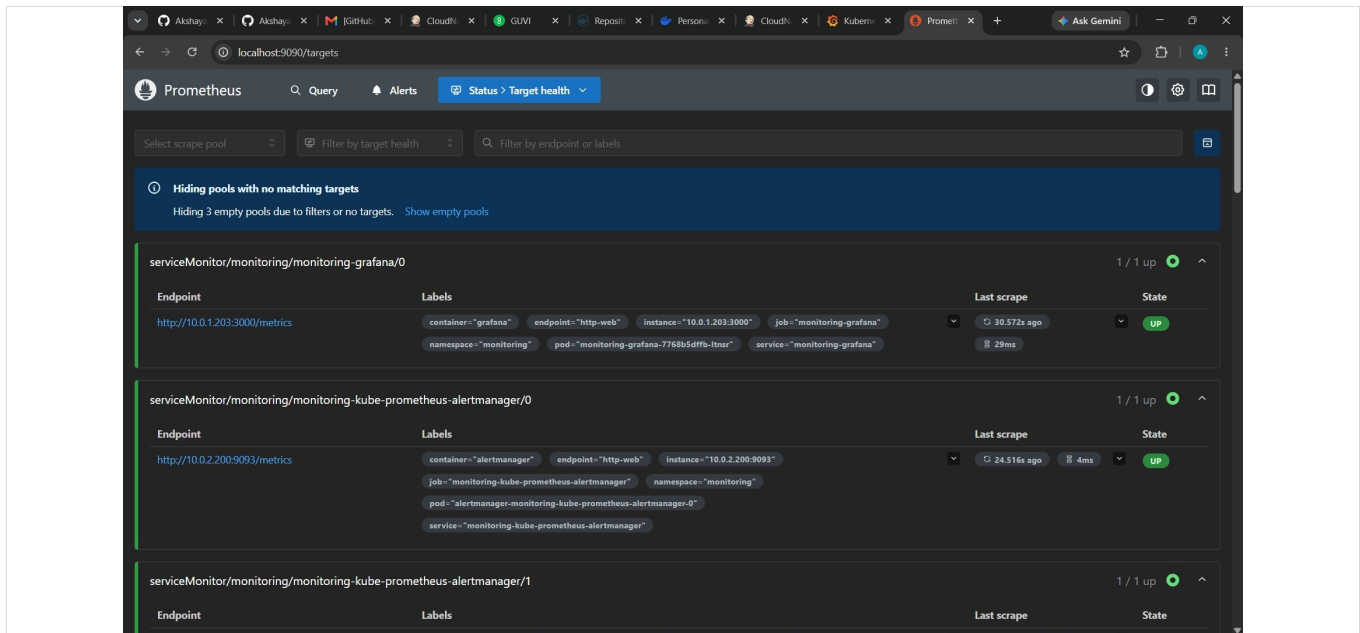


Figure 8.3 — Prometheus target health page showing Grafana and Alertmanager scrape targets UP

10. Cleanup

To avoid ongoing AWS costs, resources are torn down in a specific order — the monitoring stack (and anything it created, like the load balancer for Grafana if one existed) is removed first, then the underlying infrastructure:

6. Remove the monitoring stack

```
helm uninstall monitoring -n monitoring
kubectl delete namespace monitoring
```

7. Destroy the AWS infrastructure

```
terraform state list
terraform plan -destroy
terraform destroy
```

Why plan -destroy comes before destroy: exactly the same reasoning as terraform plan before apply — read what's about to be deleted before confirming it. This matters even more on destroy, since the action is far less reversible than creating a resource.

Verify cleanup

```
terraform state list    # should return nothing
aws eks describe-cluster --name cloudnative-eks --region ap-south-1
# should return ResourceNotFoundException
```

An empty terraform state list alone is not sufficient proof everything is gone (see Troubleshooting 10.5.4) — confirming directly against the AWS API is the more trustworthy check, especially for resources like Load Balancers that Kubernetes created outside of Terraform's own state.

11. Troubleshooting Guide

Project: Trendify cloud-native CI/CD deployment. Architecture: GitHub → Jenkins → Docker → Docker Hub → Terraform/AWS → Amazon EKS/Kubernetes → AWS Load Balancer → Trendify, with Prometheus, Grafana, and Metrics Server for monitoring.

11.2 Git and GitHub

11.2.1 Git push fails

Problem

git push is rejected or authentication fails.

Likely Cause

Wrong remote URL, an authentication problem (expired token/credentials), or a branch mismatch.

Fix

Verify the remote points to Akshaya33-Devops/CloudNative-App-Deployment and push to the main branch specifically.

Verification

```
git remote -v
git branch
git status
git push origin main
```

Interview Explanation

The repository is the single source of truth — it always contains the latest Jenkinsfile, Dockerfile, Kubernetes manifests, Terraform, and README, and Jenkins builds are only as correct as what's actually pushed here.

11.2.2 Jenkins cannot find the Jenkinsfile

Problem

A Jenkins build fails immediately with no pipeline stages executed.

Likely Cause

The Jenkinsfile is missing from the repository root Jenkins expects, or the job is configured with the wrong branch or script path.

Fix

Keep the Jenkinsfile at the Trend/ repository root and configure the job to use "Pipeline script from SCM".

Verification

```
# Check GitHub for Trend/Jenkinsfile
# Verify the Jenkins job uses branch */main and script path Jenkinsfile
# Then trigger a new Jenkins build
```

Interview Explanation

A correctly configured job prints "Obtained Jenkinsfile from git" at the very start of the console output — if that line is missing, the SCM configuration (not the pipeline logic) is the problem.

11.3 Docker

11.3.1 Docker build fails

Problem

docker build exits with an error instead of producing an image.

Likely Cause

The Dockerfile references files that don't exist relative to the build context, most commonly because dist/ is missing, or the command was run from the wrong directory.

Fix

Run the build command from the Trend/ repository root and confirm dist/ exists before building.

Verification

```
cd D:\\guvi\\CloudNative-App-Deployment\\Trend
dir
docker build -t akshayanimuthu/cloudnative-trend:dev .
docker images
```

Interview Explanation

This project ships the already-built Vite dist/ output rather than building the front end inside Docker, so a missing dist/ folder (not a Dockerfile syntax error) is the first thing to check.

11.3.2 Container starts but the application is not reachable

Problem

docker ps shows the container running, but the browser can't load the app.

Likely Cause

Port mapping is wrong, or the container isn't actually running (crashed immediately after start).

Fix

Confirm the container is running, check its logs, and use the correct port mapping for local testing.

Verification

```
docker ps
docker logs <container-name>
docker run -d -p 8081:80 akshayanimuthu/cloudnative-trend:dev
# Open http://localhost:8081
```

Interview Explanation

The container always listens on port 80 internally; host port 8081 exists purely for local convenience and is unrelated to how the app is exposed once it's running in Kubernetes.

11.3.3 Docker Hub push fails

Problem

docker push is rejected.

Likely Cause

Docker isn't logged in, the repository name doesn't match the target Docker Hub account, or the access token is invalid/expired.

Fix

Use the Docker Hub username akshayamanimuthu with a Docker Hub access token (not the account password).

Verification

```
docker login
docker images
docker push akshayamanimuthu/cloudnative-trend:dev
```

Interview Explanation

A healthy pipeline publishes both the build-number tag and the dev tag on every run — seeing only one of the two on Docker Hub is a sign the push step partially failed.

11.4 Jenkins

11.4.1 Docker Hub credential ID does not work

Problem

The Docker Push stage fails to authenticate even though the credential exists in Jenkins.

Likely Cause

The credential ID referenced in the Jenkinsfile doesn't exactly match what's configured in Jenkins — often due to a typo or a stray leading/trailing space entered when the credential was created.

Fix

Use ID dockerhub-creds with username akshayamanimuthu and a Docker Hub access token, entered with no leading or trailing spaces.

Verification

```
# Manage Jenkins -> Credentials -> Global credentials
# Confirm the ID exactly matches: dockerhub-creds
# Re-run the Docker Push stage
```

Interview Explanation

A previous run of this project failed for exactly this reason — a credential ID with a leading space looked identical in the UI but didn't match the string referenced in the Jenkinsfile. Credential IDs must match byte-for-byte.

11.4.2 Jenkins cannot run Docker

Problem

The Docker Build stage fails with a permission error when Jenkins executes docker commands.

Likely Cause

The jenkins system user isn't a member of the docker group, so it can't talk to the Docker daemon socket.

Fix

Add the jenkins user to the docker group and restart Jenkins (or the session) so the new group membership takes effect.

Verification

```
sudo -u jenkins docker ps
# Run docker ps again from inside a Jenkins build
```

Interview Explanation

Jenkins needs the same OS-level permission any user would need to run Docker commands — this is an operating-system access-control issue, not a Jenkins configuration issue.

11.4.3 Jenkins AWS commands fail

Problem

aws eks update-kubeconfig or another AWS CLI command run from a Jenkins stage returns an authorization error.

Likely Cause

Jenkins is not correctly assuming the IAM role attached to its EC2 instance (or the role itself lacks the needed permission).

Fix

Rely on the EC2 IAM role cloudnative-jenkins-role rather than static AWS access keys; confirm the instance profile is actually attached.

Verification

```
aws sts get-caller-identity
aws eks describe-cluster --region ap-south-1 --name cloudnative-eks
```

Interview Explanation

The Jenkins EC2 instance authenticates to AWS automatically through its attached IAM role — aws sts get-caller-identity is the fastest way to confirm which identity AWS thinks is making the request.

11.4.4 kubectl works partly but a command returns Forbidden

Problem

Some kubectl commands succeed (e.g. deploying pods) while others (e.g. listing nodes) return Forbidden.

Likely Cause

The IAM identity is correctly authenticated to the EKS API, but its Kubernetes-side authorization (access entry / RBAC) only grants a subset of permissions — see Section 7 for the full authentication-vs-authorization explanation.

Fix

Check the EKS access entry and the access policy assigned to the Jenkins IAM role; grant only what the pipeline actually needs.

Verification

```
kubectl auth can-i get pods
kubectl auth can-i get deployments
kubectl auth can-i get services
# Re-run the deployment and rollout verification steps
```

11.5 Terraform and AWS Infrastructure

11.5.1 terraform init fails

Problem

terraform init errors out before any plan can be generated.

Likely Cause

A provider/plugin download problem, a network issue, or the command was run from the wrong working directory.

Fix

Run Terraform from inside the terraform/ directory and confirm network access to the provider registry.

Verification

```
terraform version
terraform init
terraform providers
```

Interview Explanation

Terraform has to download the AWS provider plugin before it can plan or manage any AWS resource — this step has nothing to do with AWS credentials yet, only with Terraform's own tooling.

11.5.2 terraform plan shows unexpected changes

Problem

terraform plan reports changes that don't match what was actually intended.

Likely Cause

The local configuration, the state file, or the real AWS resources have drifted apart — often because a resource was changed manually in the AWS console.

Fix

Read the plan carefully line by line before applying anything; never apply a plan you don't fully understand, especially one showing destroys.

Verification

```
terraform state list
terraform plan
# the plan should match the intended project infrastructure exactly
```

Interview Explanation

Terraform plan works by diffing three things — configuration, state, and real infrastructure — and unexpected changes almost always mean one of those three no longer agrees with the other two.

11.5.3 EKS creation fails with IAM errors

Problem

terraform apply fails while creating the EKS cluster or node group with a permissions-related error.

Likely Cause

The EKS cluster role or node group role is missing one of the AWS-managed policies EKS requires to operate.

Fix

Confirm the EKS cluster role and node group role have the required AWS-managed policies attached, as defined in `eks_iam.tf`.

Verification

```
# Review the IAM role attachments in eks_iam.tf against the AWS error message
terraform apply
```

Interview Explanation

EKS is not a single resource — it needs IAM roles for both the control plane (to manage AWS resources on your behalf) and the worker nodes (to join the cluster and pull images), and both must exist with the right policies before the cluster can reach an Active state.

11.5.4 Terraform state is empty during cleanup**Problem**

terraform state list returns nothing, but it's unclear whether AWS resources are actually gone.

Likely Cause

The state became empty (e.g. after a destroy, or state corruption/loss) without every underlying AWS resource necessarily having been removed.

Fix

Never treat an empty Terraform state as proof by itself that every AWS resource is gone — verify directly against the AWS API.

Verification

```
terraform state list
aws eks describe-cluster --name cloudnative-eks --region ap-south-1
# Confirm the EKS cluster, EC2 instances, load balancer, and related resources are absent
```

Interview Explanation

In this project the Terraform state became empty before a full destroy was confirmed, so a direct AWS API check was used to verify the EKS cluster and EC2 instance were actually gone — state and reality can diverge, and cost-sensitive cleanup should always be double-checked against the source of truth (AWS itself).

11.6 Amazon EKS and Kubernetes**11.6.1 kubectl cannot connect to EKS****Problem**

kubectl commands time out, or connect to the wrong cluster.

Likely Cause

kubectl's context points to the wrong cluster/context, or the local kubeconfig is missing the EKS entry entirely.

Fix

Update the kubeconfig for this specific cluster and verify which context is currently active.

Verification

```
aws eks update-kubeconfig --region ap-south-1 --name cloudfnative-eks
kubectl config current-context
kubectl get pods -A
```

Interview Explanation

update-kubeconfig writes the cluster's endpoint and an AWS-IAM-based authentication method into kubectl's config — it's the bridge between an AWS identity and a Kubernetes client, and has to be re-run any time kubectl needs to target a different cluster or a fresh environment.

11.6.2 Pods are not Running

Problem

kubectl get pods shows a pod stuck in a non-Running state (Pending, CrashLoopBackOff, Error, etc.).

Likely Cause

Could be an image pull error, a misconfiguration in the manifest, insufficient node resources, or the container process itself crashing.

Fix

Use describe and logs to find the exact reason rather than guessing — the fix depends entirely on what those show.

Verification

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
```

Interview Explanation

kubectl describe surfaces Kubernetes-level scheduling and lifecycle events (e.g. image pull failures, failed readiness probes); kubectl logs surfaces what's happening inside the container itself — together they cover both layers a pod failure can come from.

11.6.3 ImagePullBackOff

Problem

A pod is stuck in ImagePullBackOff instead of starting.

Likely Cause

Kubernetes cannot pull the image/tag referenced in the manifest — usually because the tag doesn't exist yet on Docker Hub, or there's a typo in the image reference.

Fix

Verify akshayamanimuthu/cloudnative-trend:<tag> actually exists on Docker Hub and that the Deployment references the correct, already-pushed tag.

Verification

```
kubectl describe pod <pod-name>
kubectl get pods
```

Interview Explanation

The Jenkins pipeline is ordered so the image is pushed to Docker Hub before the EKS deployment stage runs — ImagePullBackOff almost always means either that ordering was broken or the manifest is pointing at a tag that was never built.

11.6.4 Deployment rollout does not complete

Problem

kubectl rollout status hangs or times out instead of reporting success.

Likely Cause

New pods are failing health checks, or Kubernetes can't schedule them (e.g. resource limits, node capacity).

Fix

Inspect the failing pods and the Deployment's own events to see exactly what's blocking the rollout.

Verification

```
kubectl rollout status deployment/trendify
kubectl get pods
kubectl describe deployment trendify
```

Interview Explanation

A rollout only completes once new pods pass their readiness checks and old pods are safely terminated — rollout status returning success is a stronger signal than kubectl apply succeeding, because apply only confirms the API accepted the manifest, not that the new version is actually healthy.

11.6.5 Service has no external address

Problem

kubectl get service trendify-service shows <pending> instead of an external hostname.

Likely Cause

The AWS Load Balancer hasn't finished provisioning yet, or the Service is misconfigured (wrong type, or a selector that doesn't match the pod labels).

Fix

Confirm the Service type is LoadBalancer and its selector (app: trendify) matches the pod labels exactly.

Verification

```
kubectl get service trendify-service
kubectl describe service trendify-service
# Wait for EXTERNAL-IP/hostname, then test it in a browser
```

Interview Explanation

A LoadBalancer-type Service asks the AWS cloud-controller-manager to provision a real ELB behind the scenes, which typically takes one to a few minutes — <pending> immediately after creation is expected, not necessarily a bug.

11.7 Helm, Metrics Server, Prometheus and Grafana

11.7.1 helm command not found on Windows

Problem

PowerShell reports helm is not recognized after installation.

Likely Cause

Helm was installed, but its install directory was never added to the current session's PATH.

Fix

Add the Helm installation directory to PATH (in this project, the WinGet installation directory) and open a new terminal session.

Verification

```
helm version
```

Interview Explanation

Helm is what installs the kube-prometheus-stack chart in one command instead of dozens of hand-written manifests — a PATH issue here is purely an OS/shell problem, not a Kubernetes or Helm configuration problem.

11.7.2 kubectl top nodes does not work

Problem

kubectl top nodes returns an error instead of resource usage numbers.

Likely Cause

Metrics Server is either not installed or its pod isn't Ready yet.

Fix

Install Metrics Server and wait for its pod to reach the Running state before retrying.

Verification

```
kubectl get pods -A | findstr metrics-server  
kubectl top nodes
```

Interview Explanation

kubectl top is a thin client on top of the Metrics API, which only exists once Metrics Server is running and has completed its first scrape cycle — it has nothing to do with Prometheus, which is a separate, independent metrics path.

11.7.3 Prometheus targets are down

Problem

The Prometheus UI shows one or more scrape targets as DOWN instead of UP.

Likely Cause

The target's pod or Service is unhealthy, unreachable, or was recently restarted and hasn't re-registered yet.

Fix

Check the specific failing target's pod and Service directly, rather than assuming the whole monitoring stack is broken.

Verification

```
kubect1 get pods -n monitoring
# Open Prometheus -> Status -> Targets
# Targets should report UP
```

Interview Explanation

Prometheus scrapes each exporter independently, so one DOWN target (e.g. a restarted Grafana pod) doesn't mean Prometheus itself is broken — it means investigation should start at that specific target's pod, not at the monitoring stack as a whole.

11.7.4 Grafana does not open on localhost

Problem

`http://localhost:3000` doesn't load after starting a port-forward.

Likely Cause

The port-forward isn't actually running, the Grafana pod isn't Ready, or local port 3000 is already in use by something else.

Fix

Confirm the Grafana pod and Service exist and are healthy, then re-run the port-forward.

Verification

```
kubect1 get pods -n monitoring
kubect1 get svc -n monitoring
kubect1 port-forward -n monitoring svc/monitoring-grafana 3000:80
# Open http://localhost:3000
```

12. Testing & Verification Checklist

A practical, layer-by-layer checklist to confirm the whole system is healthy — useful both after a fresh deployment and as a quick sanity check before a demo or interview walkthrough.

#	Check	Command / Action	Expected Result
1	Source control is current	git status / git log -1	Working tree clean; latest commit matches what was intended to deploy.
2	Image builds locally	docker build -t ...	Build completes with no errors; docker images shows the new tag.
3	Image runs locally	docker run -d -p 8081:80 ...	App loads at http://localhost:8081.
4	Infrastructure matches config	terraform plan	No unexpected changes; plan is empty or matches intended edits.
5	EKS cluster is Active	aws eks describe-cluster --name cloudnative-eks	status: ACTIVE
6	kubectl targets the right cluster	kubectl config current-context	Context name matches cloudnative-eks.
7	Jenkins pipeline succeeds	Run/trigger the Jenkins job	All stages green; console ends with Finished: SUCCESS.
8	Pods are healthy	kubectl get pods	trendify pods show 2/2 Running, 0 restarts (or low, expected restarts).
9	Rollout completed	kubectl rollout status deployment/trendify	"deployment "trendify" successfully rolled out"
10	Service has an external endpoint	kubectl get service trendify-service	EXTERNAL-IP/hostname is populated, not <pending>.
11	App is reachable publicly	Open the ELB DNS name in a browser	Trendify UI renders correctly.
12	Monitoring stack is up	kubectl get pods -n monitoring	All monitoring pods Running.
13	Prometheus targets are healthy	Prometheus UI -> Status -> Targets	All relevant targets report UP.
14	Grafana dashboards render	Open Grafana -> Kubernetes / Compute Resources	Live CPU/memory data displayed, no "No data" panels (aside from known exceptions).
15	Cleanup leaves nothing behind	terraform state list + AWS console spot check	State is empty and AWS confirms no leftover EKS/EC2/ELB resources.

13. Security Considerations

What this project already does well, and what a production deployment would need to add:

13.1 Already in place

- **No static AWS credentials.** Jenkins authenticates to AWS entirely through its EC2 instance's IAM role, so there are no long-lived access keys stored in Jenkins configuration, environment variables, or the Jenkinsfile.
- **Scoped Docker Hub credentials.** A Docker Hub access token, not the account password, is stored as a Jenkins credential and can be revoked independently.
- **Separate authentication and authorization for EKS.** IAM identity and Kubernetes RBAC (via the EKS access entry) are two distinct checks, which limits what a compromised identity can actually do inside the cluster even if AWS authentication succeeds.
- **Infrastructure as Code.** Every AWS resource is defined in Terraform, so there's an auditable history of what exists and why — no untracked manual console changes.
- **Isolated monitoring namespace.** The monitoring stack's RBAC and resources are separated from the application namespace.

13.2 Gaps to be aware of (and how to explain them in an interview)

- **No HTTPS/TLS.** The application and dashboards are served over plain HTTP. A production version would terminate TLS at the Load Balancer (an AWS ACM certificate on an ALB, or a cert-manager-issued certificate on an Ingress) rather than expose plaintext HTTP publicly.
- **Public subnets for everything.** Both the EKS nodes and the Jenkins EC2 instance sit in public subnets. A hardened design would move worker nodes into private subnets behind a NAT Gateway, exposing only the Load Balancer publicly.
- **Broad IAM policy scope.** AmazonEKSAAdminPolicy and similarly broad managed policies are convenient for a demo but wider than a real CI/CD identity needs — production usage should scope this down to only the verbs the pipeline actually calls (get/list/patch on specific resource types).
- **Grafana/Prometheus exposed only via kubectl port-forward.** Fine for a single operator during development, but not a real access model for a team — a production setup would put these behind their own authenticated Ingress or an identity-aware proxy.
- **No network policies.** By default, any pod in the cluster can talk to any other pod. Kubernetes NetworkPolicy resources would restrict the monitoring namespace and the application namespace to only the traffic they actually need.
- **No image scanning.** Images are pushed to Docker Hub without a vulnerability scan step in the pipeline; adding one (Docker Scout, Trivy, or Docker Hub's own scanning) would catch known CVEs before deployment.
- **Secrets in a Kubernetes Secret, unencrypted at rest by default.** The Grafana admin password lives in a standard Kubernetes Secret; enabling EKS's envelope encryption with a KMS key (or using AWS Secrets Manager / External Secrets Operator) would encrypt it at rest with a customer-managed key.

14. Future Improvements

Area	Improvement	Why it would help
Networking	Move EKS nodes to private subnets with a NAT Gateway	Reduces the public attack surface — only the Load Balancer needs a public IP.
TLS	Terminate HTTPS at the Load Balancer using an ACM certificate	Encrypts traffic between users and the app; required for any real production use.
IAM	Scope the Jenkins role down to least privilege	Limits the damage a compromised Jenkins box could do; currently broader than necessary.
Secrets management	Move credentials into AWS Secrets Manager or an External Secrets Operator	Centralizes rotation and auditing instead of relying on Kubernetes Secrets and Jenkins credentials alone.
Autoscaling	Add a Horizontal Pod Autoscaler and Cluster Autoscaler (or Karpenter)	Lets the app and the node group scale automatically with real traffic instead of a fixed replica count.
GitOps	Introduce Argo CD or Flux instead of Jenkins directly running kubectl	Separates "build" from "deploy" — deployment becomes a Git-driven, auditable, self-healing process.
Image security	Add a vulnerability scan stage (Trivy/Docker Scout) before push	Catches known CVEs in base images or dependencies before they reach production.
Testing	Add automated tests (unit/integration/smoke) as a pipeline stage before deploy	Currently the pipeline builds and ships without any automated correctness check on the app itself.
Observability	Add alerting rules and routes in Alertmanager (Slack/email)	The stack collects metrics today but doesn't yet notify anyone when something goes wrong.
Multi-environment	Add separate dev/staging/prod namespaces or clusters	Currently everything deploys straight to one environment; staging would catch issues before they reach users.

15. Conclusion

CloudNative-App-Deployment demonstrates a complete, self-contained DevOps lifecycle: source control, infrastructure as code, containerization, automated CI/CD, managed Kubernetes orchestration, and observability, all wired together with security-conscious defaults like IAM-role-based authentication instead of static credentials. Every piece exists to solve a specific, real problem — reproducibility, traceability, automatic recovery, or visibility — rather than being included for its own sake.

The project is deliberately built so that the entire environment can be destroyed and rebuilt from the Git repository alone, which is itself the strongest evidence that the infrastructure-as-code and CI/CD automation are doing their job. The gaps documented in Section 13 are not oversights so much as an honest map of where a demo project ends and a production-grade system would begin — being able to name them clearly is as valuable, in an interview setting, as having built the working pipeline in the first place.

16. Screenshot Reference Index

All screenshots are embedded directly beneath the topic they illustrate throughout this document (and are also stored in the repository's screenshots/ folder). This index is a quick lookup mapping each figure number to the section it appears in.

Figure	Screenshot	Section
1.1	GitHub Repository	1. Project Overview
6.1	Dockerfile	6.2 Containerize the Application
6.2	Docker Hub Image	6.2 Containerize the Application
6.3	Terraform Infrastructure	6.3 Provision AWS Infrastructure
6.4	Jenkins Credentials	6.4 Configure Jenkins
6.5	EKS Cluster	6.5 Deploy to Amazon EKS
6.6	Kubernetes Deployment	6.5 Deploy to Amazon EKS
6.7	Kubernetes Pods	6.5 Deploy to Amazon EKS
6.8	Kubernetes Service	6.5 Deploy to Amazon EKS
6.9	Jenkins Pipeline	6.6 CI/CD Pipeline
6.10	Kubernetes Monitoring	6.7 Monitoring
7.1	Live Trendify Application	8. Accessing the Application
8.1	Grafana Dashboard Overview	9.1 Grafana
8.2	Grafana Dashboard	9.1 Grafana
8.3	Prometheus Up Query	9.2 Prometheus

17. Project Links

- **Repository:** github.com/Akshaya33-Devops/CloudNative-App-Deployment
- **Docker Hub:** hub.docker.com/r/akshayanimuthu/cloudnative-trend

18. Author

Akshaya

- **GitHub:** Akshaya33-Devops
- **Docker Hub:** akshayanimuthu
- **Project Repository:** github.com/Akshaya33-Devops/CloudNative-App-Deployment